DA3408 - AIOps Lab

Ayush Kanojiya DA24B037

August 2026

1 Question 1

1.1 Identifying categories

- **a. Entanglement (CACE)**: Changing the rounding logic for Estimated Delivery time affected the entire restaurant-recommendation model, because tightly coupled feature interactions mean a change to any single input can ripple through the whole ML model.
- **b. Undeclared Consumers**: The marketing team silently reads the model's raw output table without the ML team's knowledge, creating invisible dependencies that can cause unintended breaking changes when the schema or model changes.
- **c. Configuration & Glue-Code Debt**: The training pipeline is a chain of 14 undocumented shell scripts with no orchestration tool — a Pipeline Jungle that makes the system fragile, undocumented, hard to maintain, and hard to reproduce.

1.2 Mitigation

MLflow Experiment Tracking

- Systematically records metrics, parameters, and outputs during model runs to avoid a reproducibility crisis.
- **Architecture**: Tracking Server (UI/API), Backend Store (Metadata DB), Artifact Store (Models/Plots Storage), Model Registry.
- **Core elements**: Parameters (static inputs, `log_param()`); Metrics (numerical values over epochs, `log_metric()`); Artifacts (output files/model weights, `log_artifact()`); Tags (key-value labels, `set_tag()`).
- **Execution**: runs wrapped with `with mlflow.start_run():`.

2 Question 2 — Applied: MLflow Experiment Comparison

Setup: Six experiments using an `MLPClassifier` on MNIST (via `sklearn.datasets.fetch_openml`), varying **hidden layer size** and **learning rate**, logged to MLflow under `mnist-mlp-comparison`.

2.1 Run-Comparison Table

Run	Hidden Layers	LR	Train Acc.	Train Loss	Val. Acc.
mlp-mnist-run-1	(32,)	0.001	0.98	0.02	0.93
mlp-mnist-run-2	(32,)	0.01	0.99	0.00	0.93
mlp-mnist-run-3	(64,)	0.001	0.99	0.01	0.93
mlp-mnist-run-4	(64,)	0.01	0.99	0.00	0.94
mlp-mnist-run-5	(128, 64)	0.001	0.99	0.00	0.93
mlp-mnist-run-6	(128, 64)	0.01	0.99	0.25	0.94

2.2 Written Analysis

Best-performing run: **mlp-mnist-run-6** (hidden=(128,64), lr=0.01), val_accuracy 0.94, tied with run-4 (hidden=(64,), lr=0.01). Both top runs used the higher learning rate, suggesting faster learning helped convergence within the 100-epoch budget more than architecture size alone.

Overfitting evidence is mild: train_accuracy plateaus at 0.98–0.99 while val_accuracy sits lower at 0.93–0.94 across all runs — a normal, modest gap. The clearest instability is in run-6, whose `train_loss_curve` drops sharply for ∼10 steps then rises again after step ∼30, ending at loss 0.25 vs. near-zero elsewhere — likely the (128,64) architecture combined with lr=0.01 causing the optimizer to overshoot; did not hurt val_accuracy but signals reduced stability.

Comparing hyperparameters: lr=0.01 runs (2,4,6) reach 0.93/0.94/0.94 val_accuracy vs. lr=0.001 runs (1,3,5) at 0.93/0.93/0.93. Hidden layer size alone shows almost no effect on val_accuracy but interacts with lr to cause run-6's instability. **Learning rate had the larger effect on final validation performance**; hidden layer size mattered more for training stability.

2.3 MLflow Logging Code

```
mlflow.log_param("hidden_layer_sizes", cfg["hidden_layer_sizes"])
mlflow.log_param("learning_rate_init", cfg["learning_rate_init"])
mlflow.log_param("max_iter", 100); mlflow.log_param("dataset", "MNIST")
mlflow.log_param("model_type", "MLPClassifier")
mlflow.log_metric("train_accuracy", train_accuracy)
mlflow.log_metric("val_accuracy", val_accuracy)
mlflow.log_metric("train_loss", final_train_loss)
mlflow.log_metric("n_iter", model.n_iter_)
for epoch, loss in enumerate(model.loss_curve_):
    mlflow.log_metric("train_loss_curve", loss, step=epoch)
```